Week – 12 Practical

1.Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

▢vedadripinninti@MSI123 + ▼

GNU nano 8.7.1 src/producer_consumer.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 0; i < ITEMS; i++) {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg) {
    int item;

    for (int i = 0; i < ITEMS; i++) {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);
```

▢vedadripinninti@MSI123 +

GNU nano 8.7.1

```c
        item = buffer[out];
        out = (out + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
    }

    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t producer_thread, consumer_thread;
    clock_t start, end;

    if (argc != 2) {
        printf("Usage: %s buffer_size\n", argv[0]);
        return 1;
    }

    buffer_size = atoi(argv[1]);

    if (buffer_size <= 0) {
        printf("Buffer size must be greater than 0.\n");
        return 1;
    }

    buffer = malloc(buffer_size * sizeof(int));

    if (buffer == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    sem_init(&empty, 0, buffer_size);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    start = clock();

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    end = clock();

    double time_taken =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("Buffer size: %d\n", buffer_size);
    printf("Items produced/consumed: %d\n", ITEMS);
    printf("Time taken: %.6f seconds\n", time_taken);

    if (time_taken > 0)
        printf("Throughput: %.2f items/second\n",
               ITEMS / time_taken);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    free(buffer);

    return 0;
}
```

Output:

```
vedadripinninti@MSI:~/Skill/Week-12$ nano src/producer_consumer.c

vedadripinninti@MSI:~/Skill/Week-12$ gcc src/producer_consumer.c -o bin/producer_consumer -pthread
vedadripinninti@MSI:~/Skill/Week-12$ ./bin/producer_consumer 5
Buffer size: 5
Items produced/consumed: 100000
Time taken: 1.019230 seconds
Throughput: 98113.28 items/second
```

2. Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

▢vedadripinninti@MSI123 + ▼

GNU nano 8.7.1 src/deadlock.c *

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

pthread_barrier_t barrier;

void *thread1(void *arg) {
```

▢vedadripinninti@MSI123 +

GNU nano 8.7.1

```c
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_barrier_init(&barrier, NULL, 2);
```

```
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Resource 1 acquired\n");

    pthread_barrier_wait(&barrier);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Resource 2 acquired\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg) {
    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Resource 2 acquired\n");

    pthread_barrier_wait(&barrier);

    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Resource 1 acquired\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);
```

```
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_barrier_destroy(&barrier);
    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}
```

Output:

```
vedadripinninti@OSI:~/OSI_Skill/Week-12$ nano src/deadlock.c
vedadripinninti@OSI:~/OSI_Skill/Week-12$ gcc src/deadlock.c -o bin/deadlock -pthread
vedadripinninti@OSI:~/OSI_Skill/Week-12$ ./bin/deadlock
Thread 1: Locking Resource 1
Thread 1: Resource 1 acquired
Thread 2: Locking Resource 2
Thread 2: Resource 2 acquired
Thread 2: Waiting for Resource 1
Thread 1: Waiting for Resource 2
^C
```